Algoritmer & Datastrukturer
2. november 2026

Træer

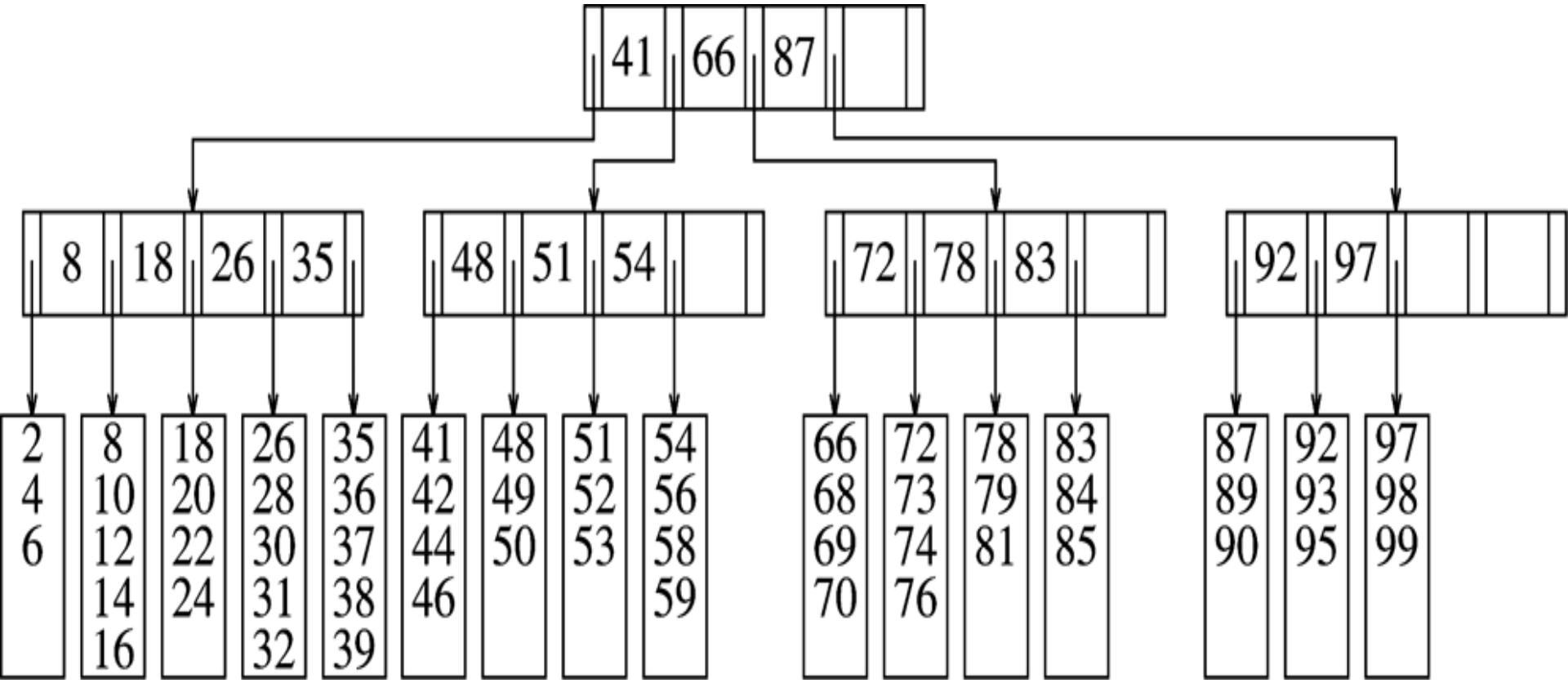
Weiss, kapitel 4

Om træer

”*Træet* er en fundamental struktur i computer science. Næsten alle operativsystemer lagrer filer i træer eller trælignende strukturer. Træer bruges også i design af compilere, tekstbehandling og søgealgoritmer (og databaser, red.).”

Mark Allen Weiss

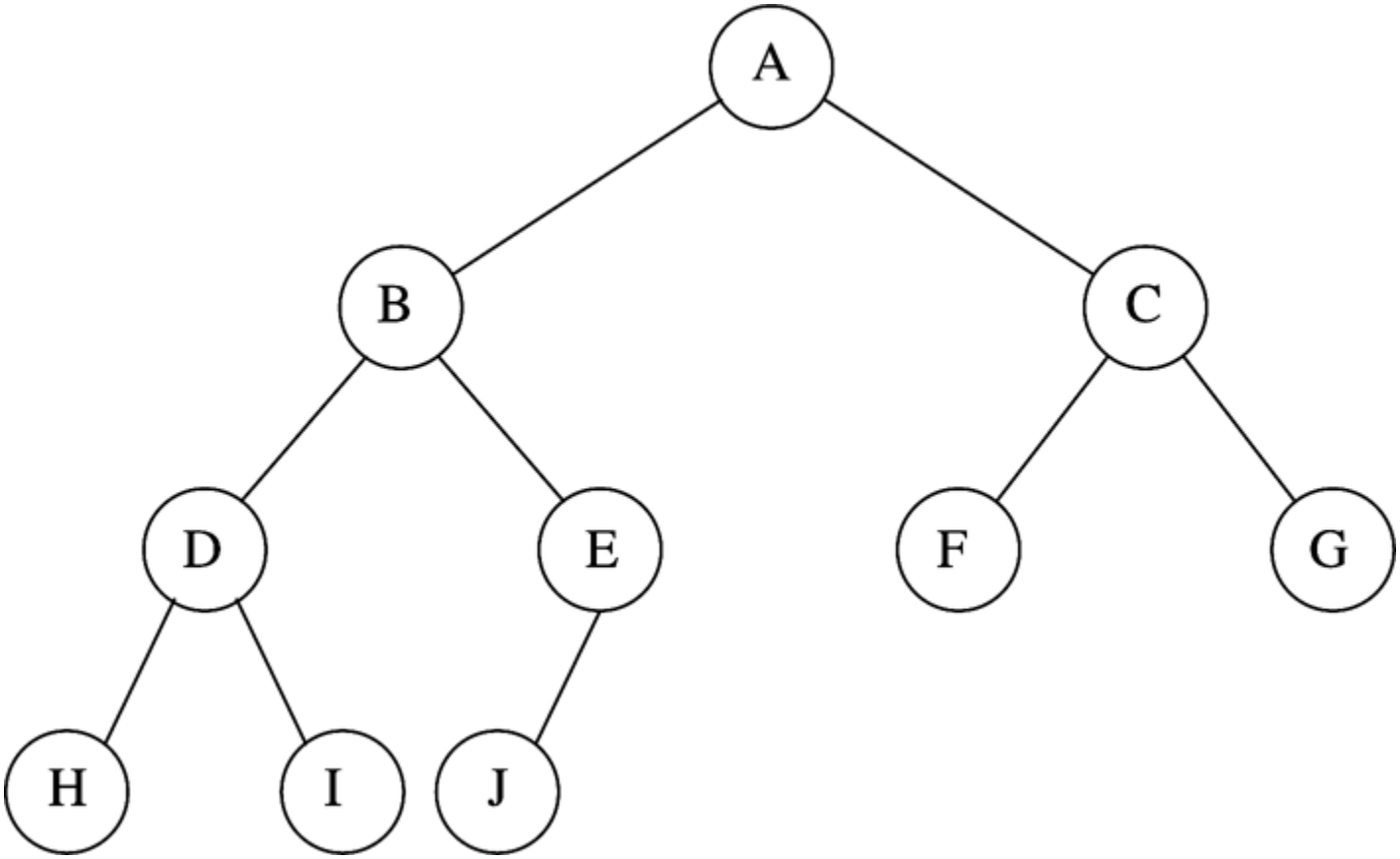
5 ordens B-træ



Introduktion

- Linkede lister med deres lineære tilgang er ikke brugbare for store datamængder.
- Vi har brug for en simpel datastruktur med en access tid på O(log N).
- En mulighed er det **binære søgetræ**.
- Collection klasserne *set* og *map* baserer sig på træer.

Eksempel på et binært træ



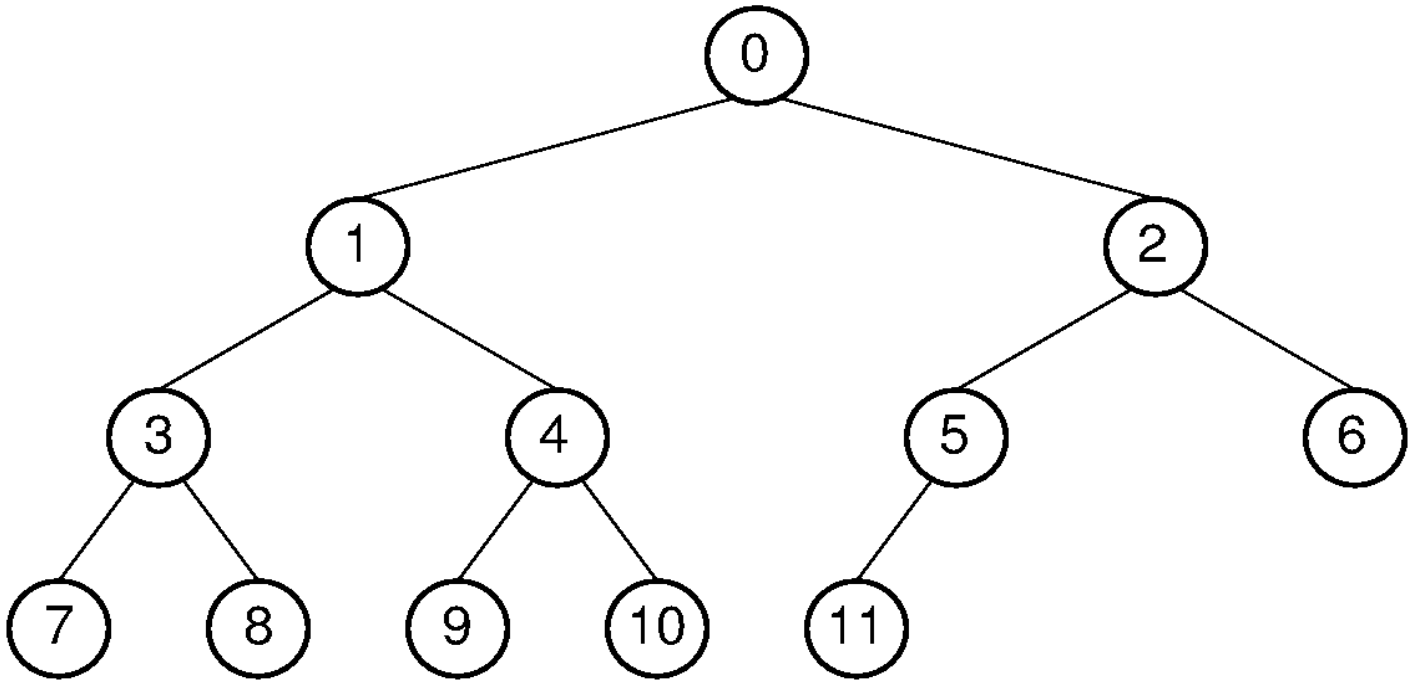
Begreber

- Dybde (depth) – afstand fra node til rod (root).
- Højde (height) – længste distance fra node til blad (leaf).

Binære træer kan være…..

- Komplette
- Perfekte
- Balancerede

Komplet binært træ

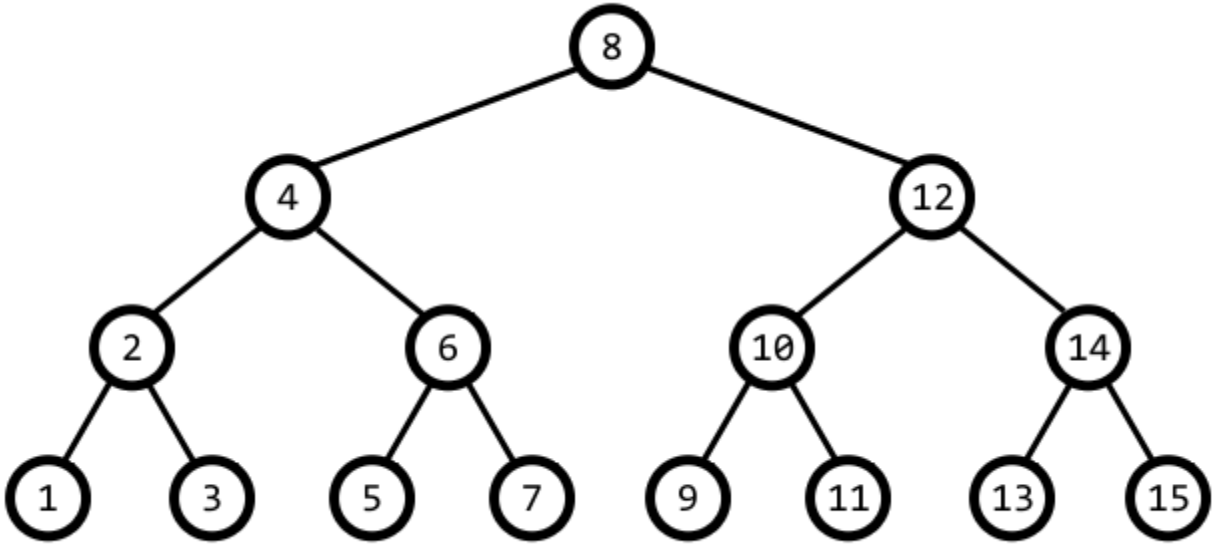


(a)

Alle niveauer er fyldt op fra venstre mod højre.

Hvis node 6 fjernes, er træet ikke længere komplet.

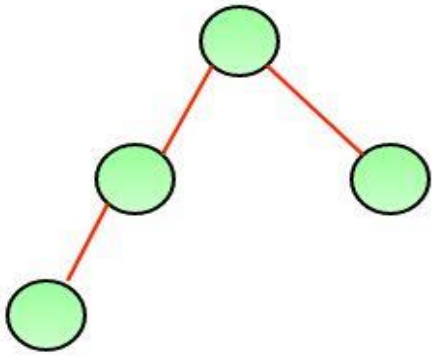
Perfekt binært træ



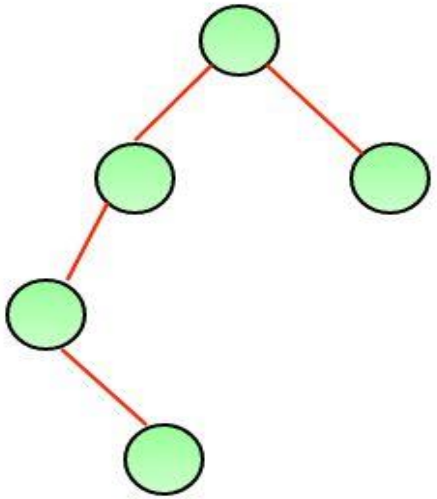
Alle niveauer er fyldt fra venstre til højre.

Antal noder N = 2^{h+1} - 1

Balanceret binært træ



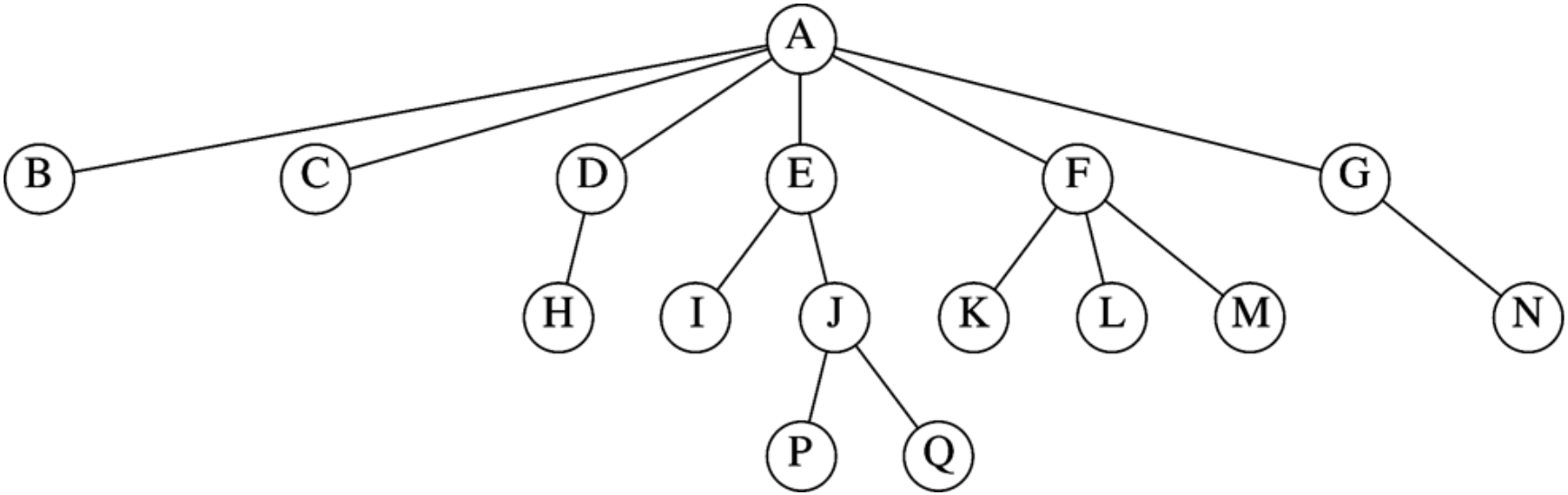
A height balanced tree

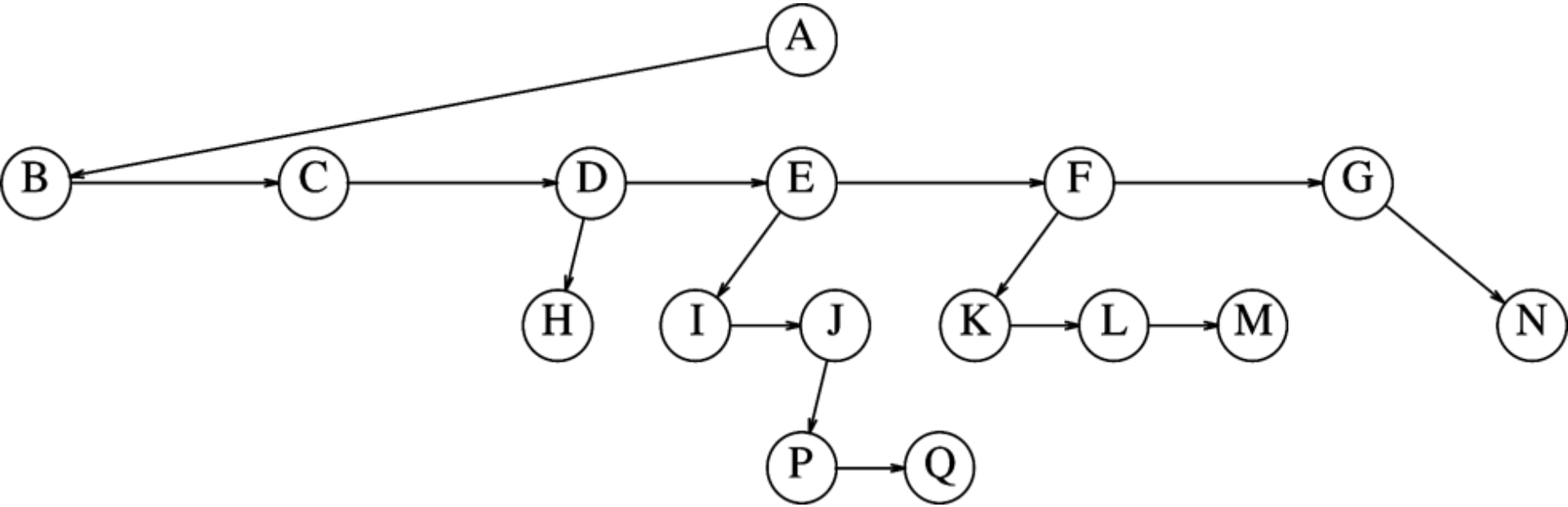


Not a height balanced tree

Højdeforskellen mellem subtræer må ikke være større end 1 (en).

n-ary trees





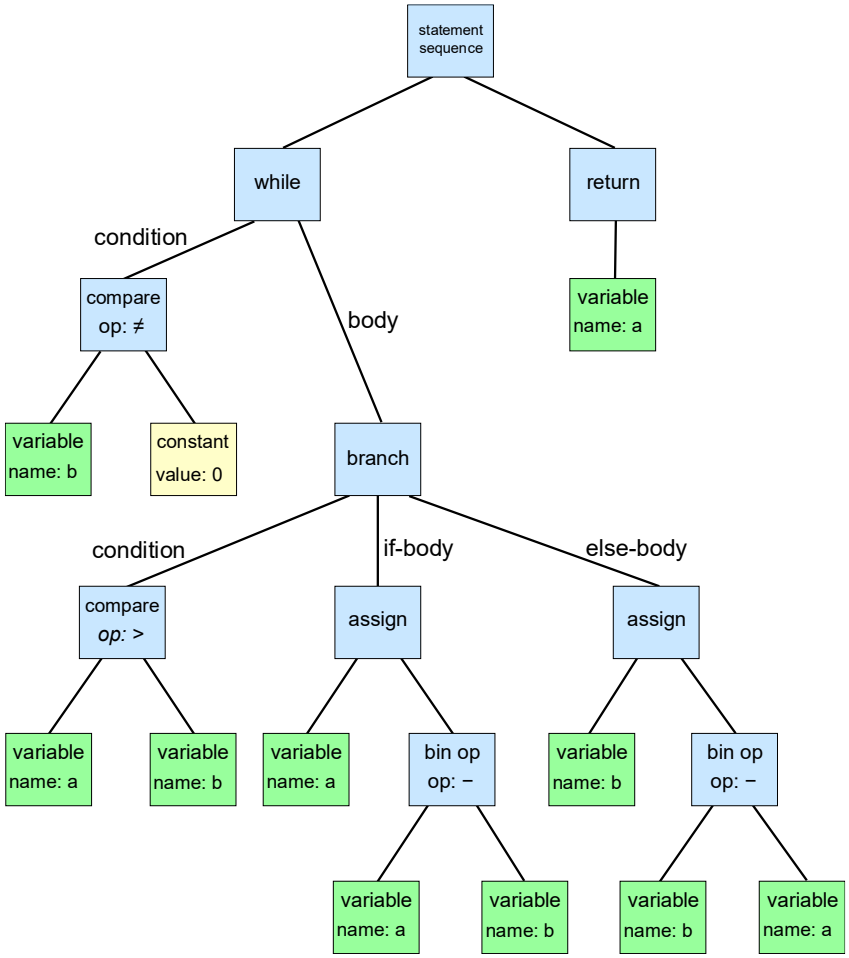
TreeNode i et ikke-binært træ

```
class TreeNode
{
    Object   element;
    TreeNode firstChild;
    TreeNode nextSibling;
}
```

Hvad kan *n-ary* trees bruges til?

- Filsystemer
- XML/HTML document trees
- Compiler syntax trees
- AI/Game trees
- Indlejrede systemer (for at spare plads)

Compiler syntax tree



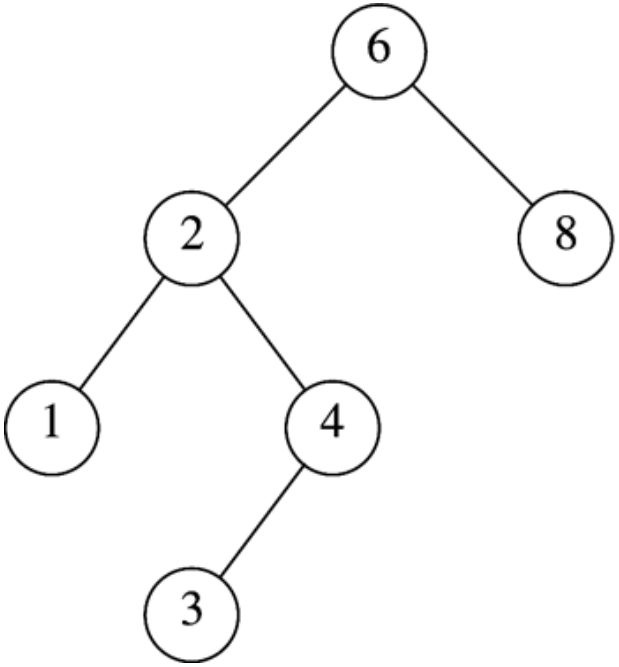
Binary Tree

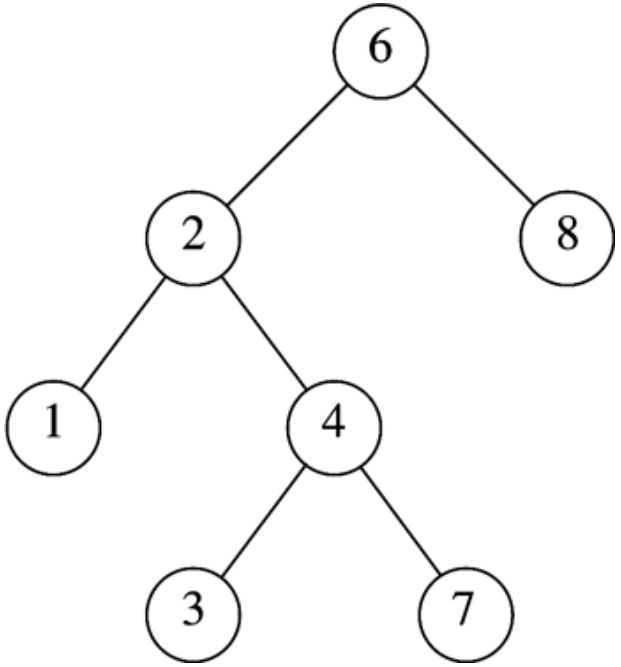
```
class BinaryNode
{
     Object     element;       // The data in the node
    BinaryNode left;          // Left child
    BinaryNode right;         // Right child
}
```

Binære træer og binære søgetræer

- Et binært træ er et træ, hvor hver node kan have 0, 1 eller 2 børn.
- Et binært søgetræ er et træ, hvor for enhver node, alle noder i det venstre subtræ er mindre end noden, og alle noder i det højre subtræ er større.
- Dvs. **ingen** dubletter.

Binære søgetræer og binære træer





Om kodeeksempler …

I skal ikke nødvendigvis kunne forklare alle kodeeksempler i pp-præsentationerne ned til mindste detalje.

Til eksamen vil I ikke blive bedt om at skrive klasser med sofistikerede constructors med anvendelse af dobbelte referencer, kompleks rekursion og andre vidtløftige fænomener. ☺

Binært søgetræ

```
// BinarySearchTree class
//
//
// ******************PUBLIC OPERATIONS*********************
// void insert( x )       --> Insert x
// void remove( x )       --> Remove x
// boolean contains( x )  --> Return true if x is present
// Comparable findMin( )  --> Return smallest item
// Comparable findMax( )  --> Return largest item
// boolean isEmpty( )     --> Return true if empty; else false
// void makeEmpty( )      --> Remove all items
// void printTree( )      --> Print tree in sorted order
```

Binært søgetræ - Node

```
    private static class BinaryNode
    {
         int element;        // The data in the node
        BinaryNode left;    // Left child
        BinaryNode right;   // Right child

       // Constructors
        BinaryNode( int theElement )
          { this( theElement, null, null ); }

        BinaryNode( int theElement, BinaryNode lt,
                                    BinaryNode rt )
          { element  = theElement; left = lt; right = rt; }

}
```

contains

```
 /**
 * Internal method to find an item in a subtree.
 * @param x is item to search for.
 * @param t the node that roots the subtree.
 * @return node containing the matched item.
 */
private boolean contains( int x, BinaryNode t )
{
    if( t == null )
        return false;

    if( x < t.element )
        return contains( x, t.left );
    else
    if( x > t.element )
        return contains( x, t.right );
    else
        return true;    // Match
}
```

findMin, findMax

```
private BinaryNode findMin( BinaryNode t )
{
    if( t == null )
        return null;
    else if( t.left == null )
        return t;
    return findMin( t.left );
}

private BinaryNode findMax( BinaryNode t )
{
    if( t != null )
        while( t.right != null )
            t = t.right;

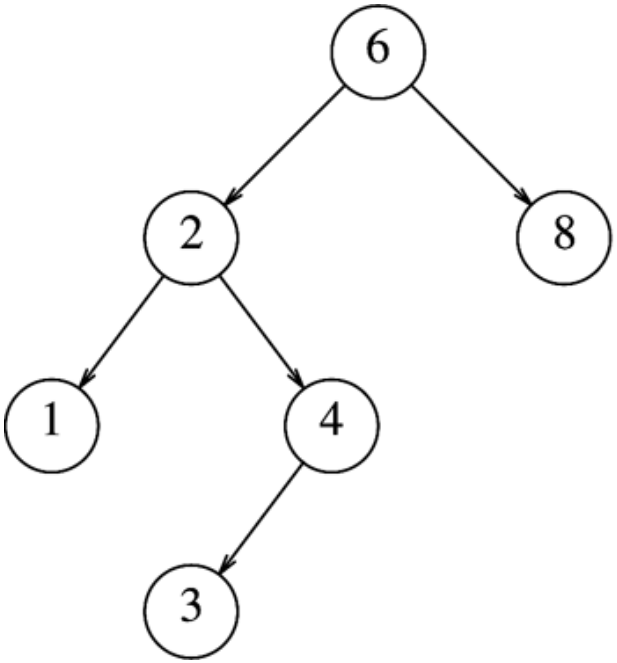
    return t;
}
```

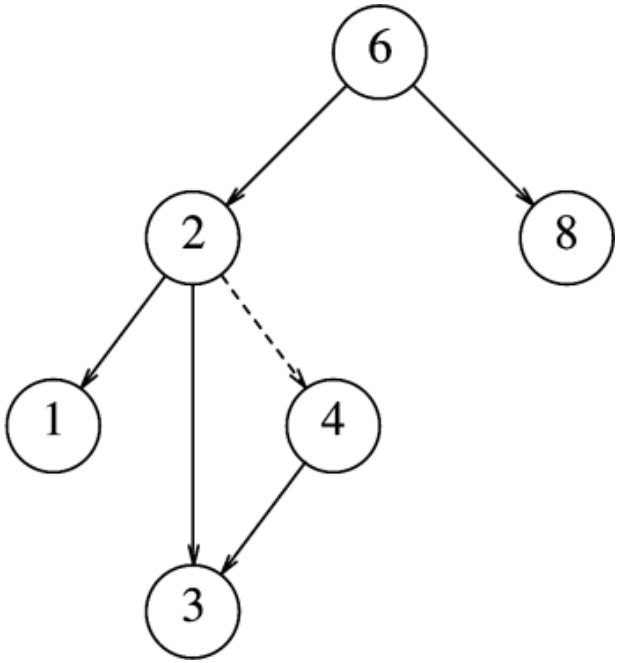
insert

```
private BinaryNode insert( int x,
                           BinaryNode t )
{
        if( t == null )
            return new BinaryNode( x, null, null );

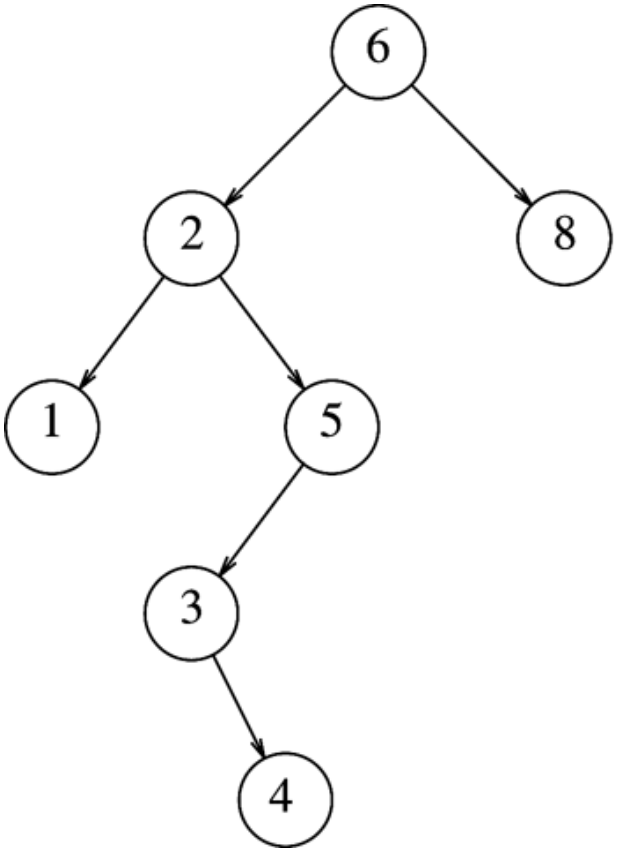
        if( x < t.element )
            t.left = insert( x, t.left );
        else
        if( x > t.element )
            t.right = insert( x, t.right );
        else
            ;  // Duplicate; do nothing
        return t;
}
```

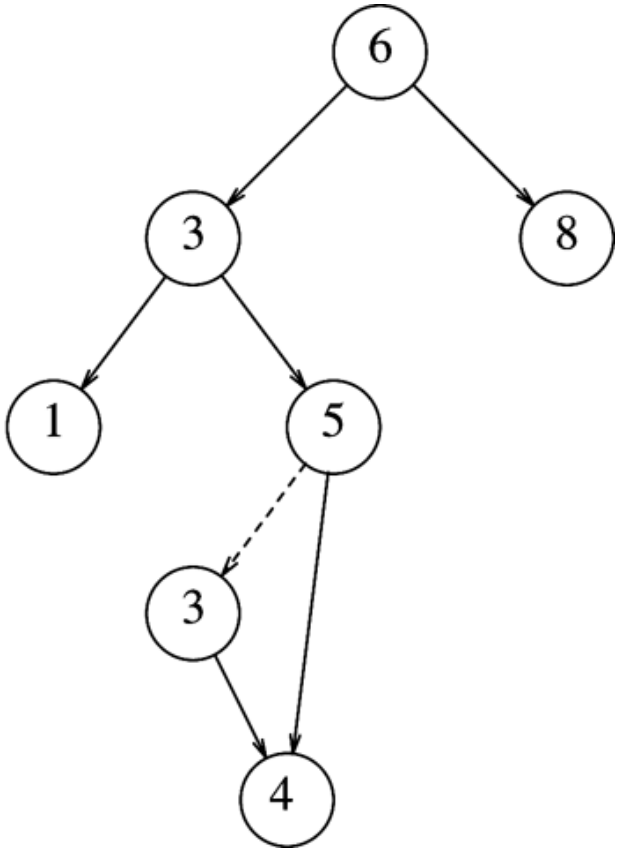
remove(4)





remove(2)





remove

```
private BinaryNode remove( int x, BinaryNode t )
{
    if( t == null )
        return t;   // Item not found; do nothing

     if( x < t.element )
        t.left = remove( x, t.left );
    else
    if( x > t.element )
        t.right = remove( x, t.right );
    else
    if( t.left != null && t.right != null ) // Two children
    {
        t.element = findMin( t.right ).element;
        t.right = remove( t.element, t.right );
    }
    else
        t = ( t.left != null ) ? t.left : t.right;
    return t;
}
```

Binært søgetræ

- Definition: *balance condition* – dybden af træet er O(log N).
- Søgetiden er så ligeledes O(log N).
- Det holder bare ikke, hvis træerne bliver højre- eller venstretunge på grund af gentagne indsættelser og sletninger.

Internal path length

Weiss:

”The sum of all the depths of all nodes in a tree is known as the internal path length.”

Andre mener, at bladene **ikke** skal tælles med, og det er derfor, det hedder *internal* path length.

ChatGPT

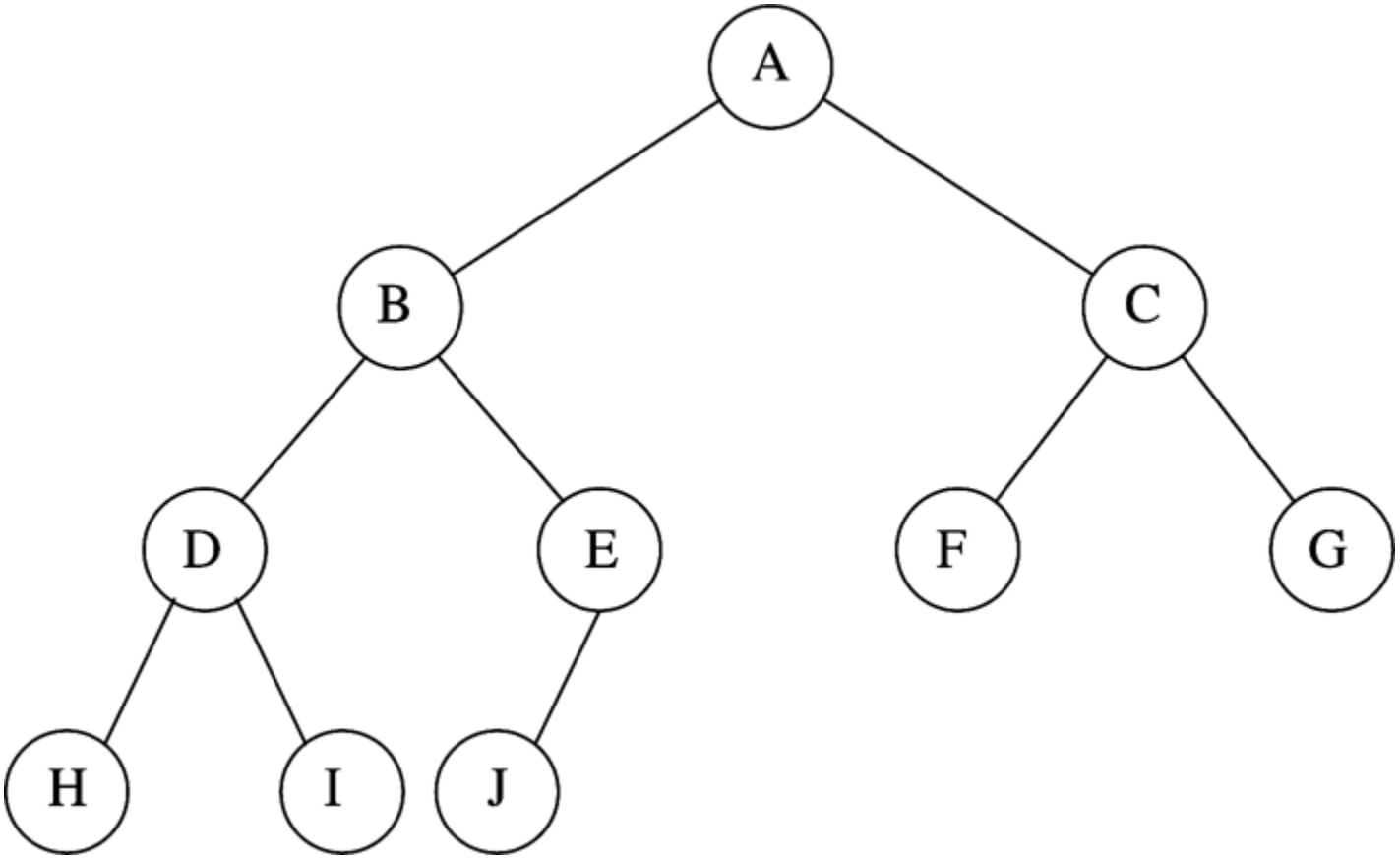
Internal Path Length (IPL) can sometimes refer to the sum of depths of all nodes in a tree, including both internal nodes and leaves, depending on the context. This broader definition is useful when talking about the general structure and efficiency of operations on a binary tree, including searches, which involve both internal nodes and leaves.

However, in other contexts, IPL is sometimes specifically defined as the sum of the depths of internal nodes only, excluding leaves, especially when we focus on the structural properties of internal nodes alone.

Vi går med Weiss og tæller bladene med!

Binært træ med internal path length = 19

2*1 (B,C)+4*2(D,E,F,G)+3*3(H,I,J)



Average case Analysis

Summen af dybderne af samtlige noder kaldes træets **internal path length** og kan beskrives på følgende måde:

$$D(N) = D(i) + D(N - i - 1) + N - 1$$

Hvor de to første elementer på højresiden er venstre og højre subtræ set fra roden.

Gennemsnitsværdien af D(N) kan beskrives som en *rekurrens,* på denne måde:

$$D(\mathrm{N}) = \frac{2}{N}\left[\sum_{j=0}^{N-1} D(j)\right] + N - 1$$

Average case analysis

Det kan (som bekendt) bevises, at

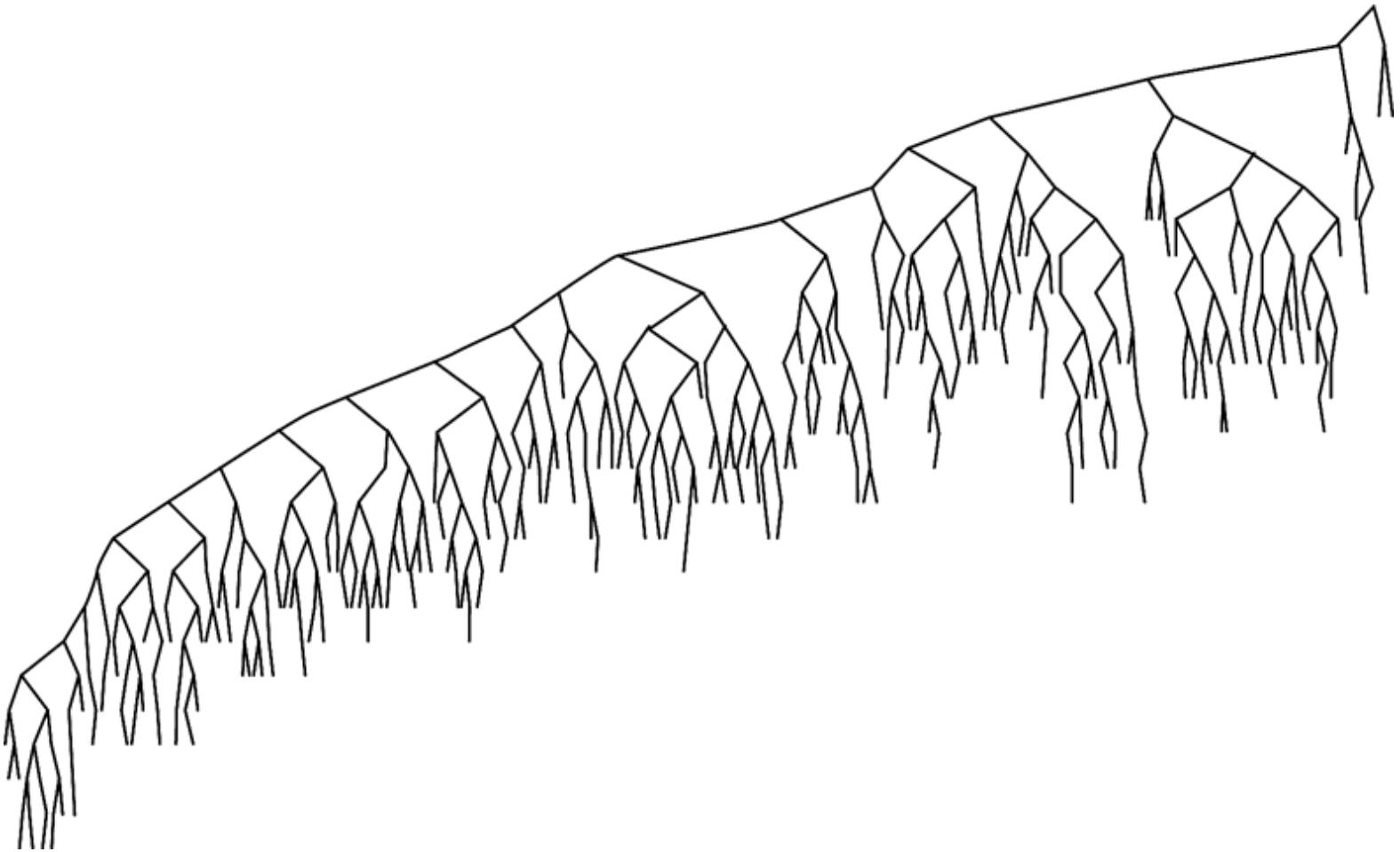
$$D(\mathrm{N}) = \frac{2}{N}\left[\sum_{j=0}^{N-1} D(j)\right] + N - 1$$

er O(N log N), hvilket betyder at den **gennemsnitlige** dybde for alle noder i et træ er O(log N).

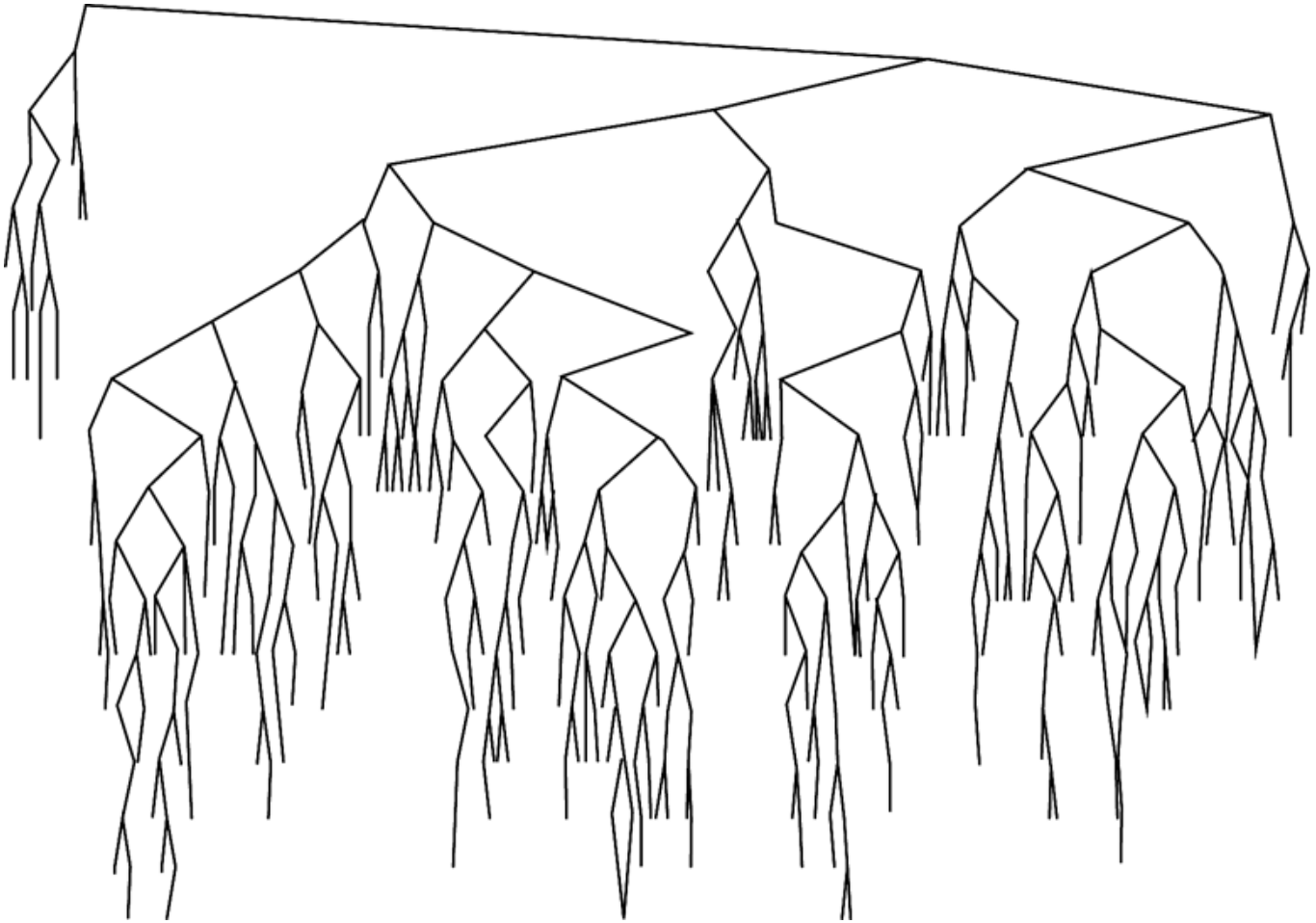
Det giver mening under antagelse af, at alle træer er lige sandsynlige og dermed, at alle indsættelses-sekvenser er lige sandsynlige.

Husk at i det perfekte træ er den **maksimale** dybde O(log N).

Et temmelig ubalanceret træ



Tilfældigt generet træ – med 500 noder – gennemsnitsdybden er 9,98 (log 500 ~ 9)



Problemerne med average case og anden analyse

Er massive. Overvej følgende:

- Er alle træer lige sandsynlige?
- Hvad hvis input er mere eller mindre sorteret?
- Hvad betyder *average* egentlig?
- Antallet af muligheder er overvældende stort.

Konklusion

Alting bliver meget nemmere, hvis træet er *balanceret*, hvilket betyder, at

- Dybden er altid tæt på O(log N)
- Eller med andre ord: for hver node er højdeforskellen på dets subtræer aldrig større end 1.
- En løsning er det såkaldte AVL-træ.
- Mere herom næste gang.

Øvelser,
som kodes på den udleverede BST-kode

1. Skriv metoder til pre-order, in-order, post-order og level order traversering af binære (søge) træer.
2. Skriv en metode, som kan beregne internal path length af et binært træ.
3. 4.31
4. Skriv en metode, som kan skabe et perfekt binært træ med højden h (>0).